

Welcome back

How was lab? Was anything confusing?

```
x = 1
x = x + 3
x
4

x = 1
x += 3
x
4

x = 1
x = x - (-3)
x
4

x = 1
x -= -3
x
4

x = 1.6
# What is x rounded?
2
2

x = 1.4
# What is x rounded?
1
1

x = 10.6
y = 10
rounded_value = 11

int(x) == y
True

x = 10.6
y = 10
```

```

decimal_only = x - y

x - y
0.5999999999999996

x = 10.5
y = int(x)
decimal_only = x - y

if decimal_only >= 0.5:
    rounded_value = y + 1
else:
    rounded_value = y

rounded_value
11

# This is a comment
x = 1
# This is another comment
x
1

n1 = 16
n2 = -2

# Goal: print n2 with the sign of n1
2

2

n1 = -16
n2 = -2

if n1 >= 0 and n2 >= 0:
    print(n2)
elif n1 >= 0 and n2 < 0:
    print(n2 * (-1))
elif n1 < 0 and n2 >= 0:
    print(n2 * (-1))
else: # n1 < 0 and n2 < 0
    print(n2)

-2

n1 = -16
n2 = -2

if (n1 >= 0 and n2 >= 0) or (n1 < 0 and n2 < 0):

```

```
    print(n2)
else:
    print(n2 * (-1))

-2
```

Review

We covered:

- Variables
- Printing
- Comments
- Conditionals

```
x = 1
x = 2
x
2

x = 1
y = 2
z = x + y
z
3

x = 1
y = 2
z = x + y

print(x)
print(y)
print(z)

1
2
3

x = 2
x += 3 # x = x + 3
x
5

# This is a comment

x = 2
y = 3
```

```
z = x * y + x # Comments can be inline with the code
```

```
z
```

```
8
```

```
# Jupyter remembers variables across cells
```

```
z
```

```
8
```

```
if True:
    print("hi")
```

```
hi
```

```
if True:
print("hi")
```

```
Cell In[23], line 2
```

```
    print("hi")
```

```
    ^
```

IndentationError: expected an indented block after 'if' statement on line 1

```
numeric_grade = 75
```

```
if numeric_grade > 90:
    letter_grade = 'A'
elif numeric_grade > 80:
    letter_grade = 'B'
elif numeric_grade > 70:
    letter_grade = 'C'
elif numeric_grade > 60:
    letter_grade = 'D'
else:
    letter_grade = 'F'
```

```
letter_grade
```

```
'C'
```

Content I forgot to teach

Special operator for booleans: `not` keyword

```
not True
```

```
False
```

```
not False
True
if not True:
    print('hi')
else:
    print('bye')
bye
if False:
    print('hi')
if True:
    print('hi')
hi
```

Reminder: if you're confused, you can test in Jupyter!

```
# Things that were confusing from earlier today
```

Lists

A list holds Python "things" *in order*. The syntax for list is to use `[thing0, thing1, thing2]`.

IMPORTANT: this is NOT THE SAME as `()`

```
# Basic list
[1, 2]
[1, 2]
# Lists can have repeated elements
[1, 2, 1]
[1, 2, 1]
# Lists can be short or long
[True]
[True]
```

```
# Lists can have the same value repeated
[True, False, True, False, True, False]

[True, False, True, False, True, False]

# List can hold multiple "types" of things
[1, True, "hi"]

[1, True, 'hi']

# You can assign lists to variables
x = [1, 2]
x

[1, 2]

[]

[]

[1.0]

[1.0]

[1, 2.0, True, "hi", [1, 2]]

[1, 2.0, True, 'hi', [1, 2]]
```

List comparisons

```
1 == 1
True

[] == []
True

[1] == []
False

[1] == [1]
True

[1, 2] != [2, 3]
True

not ([1] == [1])
[1] != [1]
```

False

```
[1] > [2]
```

False

```
[1, 2, 1] > [1, 1, 2]
```

True

```
1 > "0"
```

```
-----  
-----  
TypeError                                Traceback (most recent call  
last)  
/tmp/ipykernel_10907/3751005170.py in <module>  
----> 1 1 > "0"
```

TypeError: '>' not supported between instances of 'int' and 'str'

```
[1, 2, 1] > [1, "0", 2]
```

```
-----  
-----  
TypeError                                Traceback (most recent call  
last)  
/tmp/ipykernel_10907/964030694.py in <module>  
----> 1 [1, 2, 1] > [1, "0", 2]
```

TypeError: '>' not supported between instances of 'int' and 'str'

Addition with lists

You can "add" two lists (this is called concatenation).

```
[1, 2] + [3, 4]
```

```
[1, 2, 3, 4]
```

```
x = [1, 2]
```

```
y = [3, 4]
```

```
print(x)
```

```
print(y)
```

```
[1, 2]
```

```
[3, 4]
```

```
x + y
```

```
[1, 2, 3, 4]
```

```
[True, False] + [1, 2]
```

```
[True, False, 1, 2]
```

```
[1, 2] + 1
```

```
-----  
-----  
TypeError                                Traceback (most recent call  
last)
```

```
/tmp/ipykernel_10907/523277706.py in <module>
```

```
----> 1 [1, 2] + 1
```

```
TypeError: can only concatenate list (not "int") to list
```

```
[1, 2] + [1]
```

```
[1, 2, 1]
```

```
[1, 2] + "hi"
```

```
-----  
-----  
TypeError                                Traceback (most recent call  
last)
```

```
/tmp/ipykernel_10907/23792725.py in <module>
```

```
----> 1 [1, 2] + "hi"
```

```
TypeError: can only concatenate list (not "str") to list
```

```
[1, 2] + True
```

```
-----  
-----  
TypeError                                Traceback (most recent call  
last)
```

```
/tmp/ipykernel_10907/2579976634.py in <module>
```

```
----> 1 [1, 2] + True
```

```
TypeError: can only concatenate list (not "bool") to list
```

```
int(1.0)
```

```
1
```

```
str(1)
```

```
'1'
```

```
[True]
```

```
[True]
```

Multiplication with lists

You can also multiply lists with integers

```
"hello" * 3
'hellohellohello'

[1, 2] * 3
[1, 2] + [1, 2] + [1, 2]
[1, 2, 1, 2, 1, 2]
[1, 2] * 1.1
-----
-----
TypeError                                Traceback (most recent call
last)
/tmp/ipykernel_10907/1850234555.py in <module>
----> 1 [1, 2] * 1.1

TypeError: can't multiply sequence by non-int of type 'float'

[1, 2] * 0
[]

[] == 1
False

[1, 2] * True
[1, 2]

[1, 2] * False
[]
```

Accessing elements of a list

You can access individual elements of a list using the following syntax

```
x = [1, 2, 3, 4, 5]
x[0] # VERY IMPORTANT: list indexes start from 0
```

1

```
x(0)
```

```
-----  
-----  
TypeError                                Traceback (most recent call  
last)  
/tmp/ipykernel_10907/602412890.py in <module>  
----> 1 x(0)
```

TypeError: 'list' object is not callable

```
x = [1, 2, 3, 4, 5]  
x[1]
```

2

```
x[2]
```

3

```
x[10]
```

```
-----  
-----  
IndexError                                Traceback (most recent call  
last)  
/tmp/ipykernel_10907/1934062653.py in <module>  
----> 1 x[10]
```

IndexError: list index out of range

```
x[1.2]
```

```
-----  
-----  
TypeError                                Traceback (most recent call  
last)  
/tmp/ipykernel_10907/849164868.py in <module>  
----> 1 x[1.2]
```

TypeError: list indices must be integers or slices, not float

```
x["1"]
```

```
-----  
-----  
TypeError                                Traceback (most recent call  
last)  
/tmp/ipykernel_10907/3850214219.py in <module>  
----> 1 x["1"]
```

```
TypeError: list indices must be integers or slices, not str
```

```
x[1]
```

```
2
```

```
x[1]
```

```
2
```

Modifying lists

You can also change what is inside a list

```
x = [1, 2, 3]
```

```
x
```

```
[1, 2, 3]
```

```
x[1]
```

```
2
```

```
x[1] = 5
```

```
x
```

```
[1, 5, 3]
```

```
x
```

```
[1, 5, 3]
```

Appending to a list

You can also add to a list

```
x = [1, 2, 3]
```

```
x
```

```
[1, 2, 3]
```

```
x.append(4)
```

```
x
```

```
[1, 2, 3, 4]
```

```
x = [1, 2, 3]
x = x + [4]
x
[1, 2, 3, 4]
```

Indexing with negative numbers

You can also index with negative numbers. They count from the back

```
x = [1, 2, 3, 4, 5, 6]
x[-1]
6
x[-2]
5
x = [True, "hi", 1]
x[0]
True
[True, "hi", 1][0]
True
```

Slices

You can also create lists by taking a sublist of an existing list. This is often called a slice

```
x = [1, 2, 3, 4, 5, 6]
x
[1, 2, 3, 4, 5, 6]
x = [True, 5, "hi"]
x[2]
'hi'
x = [1, 2, 3, 4, 5, 6]
x[0:2] # [x[0], x[1]]
[1, 2]
```

```

x[0:3] # [x[0], x[1], x[2]]
[1, 2, 3]
x[0:4]
[1, 2, 3, 4]
x[0:10]
[1, 2, 3, 4, 5, 6]
x[0:]
[1, 2, 3, 4, 5, 6]
x[0:2]
x[:2]
[1, 2]
x[-3:-1]
[4, 5]
x[-3:]
[4, 5, 6]
x = [True, 5, "hi", 3.14]
x[0:2]
[True, 5]
x = [True, 5, "hi", 3.14]
x[1:3]
[5, 'hi']
x = [True, 5, "hi"]
x[1:]
[5, 'hi']

```

Slicing with a step

You can also slice with a *step*. It specifies to take every *n*th element

```

x = [1, 2, 3, 4, 5, 6, 7, 8, 9]
x

```

```
[1, 2, 3, 4, 5, 6, 7, 8, 9]
# x[start:end:step]
# x[start]

x[1:5:2]
[x[1], x[3]]

[2, 4]

x[0:5:2] # [x[0], x[2], x[4]]

[1, 3, 5]

x[1:5:2] # [x[1], x[3]]

[2, 4]

x[2:8:3] # [x[2], x[5]]

[3, 6]
```

Indexing and slicing strings

Similar to how you can index and slice lists, you can also index and slice strings

```
x = "Hello!"
x

'Hello!'

x[0]

'H'

x[-1]

'!'

x[0:5] # x[0] x[1] x[2] x[3] x[4]

'Hello'
```

Important note: you CANNOT modify strings

```
x = "hello"
x

'hello'
```

```
x[3] = "bye"
```

```
-----  
-----  
TypeError                                Traceback (most recent call  
last)  
/tmp/ipykernel_10907/3052838824.py in <module>  
----> 1 x[3] = "bye"  
  
TypeError: 'str' object does not support item assignment
```

Lengths

Lists and strings have *lengths*. We can determine the length of a list or string with `len` as follows:

```
x = [1, 2, 3]  
len(x)  
  
3  
  
y = "hello !"  
len(y)  
  
7
```

Dictionaries

Dictionaries also hold "things" in Python. However, every "thing" it holds is associated with a *key*. The "things" associated with the keys are called *values*.

A key can be any basic type (everything we've learned about so far except a list).

A value can be anything we've learned about so far.

The syntax for dictionaries can be a bit confusing. Let's walk through it

```
# This creates a dictionary  
country_capitals = {  
    # key: value,  
    1: 2,  
    "Ethiopia": "Addis Abada",  
    "Canada": "Ottawa",  
    "Iran": "Teheran",  
    "Turkey": "Ankara",  
}  
country_capitals
```

```

{1: 2,
 'Ethiopia': 'Addis Abada',
 'Canada': 'Ottawa',
 'Iran': 'Teheran',
 'Turkey': 'Ankara'}

# This accesses a dictionary
country_capitals["Ethiopia"]

'Addis Abada'

# Dictionaries can have keys of different types
dictionary = {
    2: "hi",
    "key": "value"
}

dictionary[2]

'hi'

dictionary["key"]

'value'

# Dictionaries can have values of different types
dictionary = {
    2: "value",
    "key": 2,
}

dictionary["key"]

2

# You must have the comma after items!
dictionary = {
    2: "value",
    True: False
    "key": 2
}

File "/tmp/ipykernel_10907/2313417585.py", line 5
    "key": 2
    ^
SyntaxError: invalid syntax

dictionary = {
    2: "value",
    "key": 3,
}

```

```
dictionary[3]
```

```
-----  
-----  
KeyError                                Traceback (most recent call  
last)  
/tmp/ipykernel_10907/1049092103.py in <module>  
      4 }  
      5  
----> 6 dictionary[3]  
  
KeyError: 3
```

Modifying dictionaries

In addition to declaring a dictionary, you can also modify a dictionary as follows

```
x = {  
    2: 3,  
    "key": "value"  
}
```

```
x[2]
```

```
3
```

```
x[2] = "new value"
```

```
x[2]
```

```
'new value'
```

```
x
```

```
{2: 'new value', 'key': 'value'}
```

```
x[4]
```

```
-----  
-----  
KeyError                                Traceback (most recent call  
last)  
/tmp/ipykernel_10907/2374535372.py in <module>  
----> 1 x[4]  
  
KeyError: 4
```

```
x[4] = 5
```

```
x[4]
5
x
{2: 'new value', 'key': 'value', 4: 5}
```

Length of dictionaries

You can also compute the length of a dictionary in the same way

```
x = {
    2: 3,
    "key": "value"
}
len(x)
2
```

Checking if a key is in a dictionary

There is a special Python keyword to check if a key is in a dictionary. It is the `in` keyword. You can use it as follows:

```
x = {
    2: 3,
    "key": "value"
}
2 in x
True
3 in x
False
x = {
    2: 3,
    2: 4,
    2: 5,
    "key": "value",
}
x[2]
```

Advanced topics: motivation for while loops

Often times, we want to repeat a computation a number of times.

```
x = [32587, 18, 43, 985, 274398]
print(x[0])
print(x[1])
print(x[2])
print(x[3])
print(x[4])
```

Advanced topic: while loops

We can do this repeated computation with while loops!

We'll start with some examples before defining the syntax

```
x = [32587, 18, 43, 985, 274398]

index = 0
while index < len(x):
    print(x[index])
    index += 1
```

While loop syntax

The syntax for while loops is:

```
while <condition>:
    <expression>
```

As with `if` statements, the `<condition>` can be anything that evaluates to a boolean. The `<expression>` can be complex.

IMPORTANT: THE WHILE LOOP MUST TERMINATE

```
# WARNING
# WARNING
# WARNING
# DO NOT RUN THIS

while True:
    print('hi')

# WARNING
# WARNING
# WARNING
# DO NOT RUN THIS

num = 0
while num < 10:
    print(num)
```